Guardrail: Irreversibility-Aware Action Gating for Human-in-the-Loop Control of Autonomous AI Agents

Devansh Verma, Muskan Gautam
Department of Computer Science and Engineering
ABES Engineering College, Ghaziabad, India
Project Guide: Ms. Akshita

***Abstract*—LLM-based agents no longer only answer questions; they issue refunds, change databases, and push code. Yet most deployed safeguards decide when to involve a human from a single number, the model's confidence, compared against one global threshold. Confidence indicates how likely an action is to be wrong, but not how much damage follows if it is. We present the design of Guardrail, a gating layer between an agent and its tools that estimates the expected damage of each proposed tool call, the probability that the call is wrong multiplied by the predicted cost of undoing it, and escalates the call to a human when this exceeds the cost of interrupting the operator, subject to a cap on interruptions. The design combines an irreversibility taxonomy, a sandbox that labels actions by executing and reversing them, a conformally calibrated undo-cost regressor, a detector for context-dependent and compositional irreversibility, and a budgeted escalation policy exposed through a Model Context Protocol proxy. We also specify a pre-registered evaluation against six baselines on tau-bench and SWE-bench. A synthetic analysis under stated assumptions shows that, at one interruption per task, the consequence-aware rule reduces irreversible errors from 6.81% to 2.60% of tasks relative to a perfectly calibrated confidence gate. The same analysis exposes a limit: about a quarter of irreversible actions look cheap when examined individually and leave an error floor that no per-call estimator can remove. These figures describe the decision rule only; the performance of the full system has not yet been measured.**

***Index Terms*—AI agents, human-in-the-loop, action gating, irreversibility, conformal prediction, quantile regression, tool use, Model Context Protocol, AI safety.**

I. Introduction

Until recently, the worst outcome of a language model error was an incorrect answer, which a reader could recognize and discard. Agents built on large language models (LLMs) now act on a user's behalf: they send email, issue refunds, modify database records, and write and deploy code. An incorrect sentence is easy to ignore. An incorrect action changes the state of the world, and in some cases it cannot be reversed.

This authority is being extended to systems that still fail frequently. In the original WebArena study, the best agent completed 14.41% of its tasks, against 78.24% for human participants [1]. A meaningful per-step error rate should therefore be treated not as a temporary limitation but as a condition that any serious design must plan around from the start.

A further weakness concerns where protection lives. Safeguards are often written as instructions in the prompt, and nothing in the execution path enforces them. Such a rule exists on paper but is not enforced where the damage occurs.

When deployed agents do stop to consult a person, the decision almost always rests on confidence [2]: the model estimates how certain it is, and a human is involved only when that estimate falls below a fixed cutoff. This answers the question "how likely am I to be wrong?" but not the more pressing one, "what happens if I am?"

Consider an agent that is 80% certain about two actions. One renames a scratch file; the other approves an irreversible transfer of funds. Under a confidence threshold, both receive identical treatment. Adjusting the cutoff does not resolve this, because a model's certainty and an action's potential damage are, in general, unrelated quantities (Fig. 1), and a single threshold cannot capture two independent dimensions. Static allow and deny lists fail for a different reason. They judge an action by its type alone, whereas reversibility often depends on context. Sending a message is usually harmless; sending the wrong message to a regulator cannot be undone.

We argue that the quantity a gate should reason about is the expected cost of undoing an action, and we design a system, Guardrail, that estimates this cost before execution and converts it into a budgeted escalation decision. The specific contributions are:

1. A formulation of agent escalation as an expected-value decision, $P_{err} \cdot U > C_{int}$, together with an argument that no confidence threshold can reproduce it whenever confidence and undo cost are not perfectly dependent.
2. A system design comprising an irreversibility taxonomy, a rollback-instrumented sandbox that produces empirically measured undo-cost labels, a conformally calibrated undo-cost regressor, a contextual and compositional irreversibility detector, and a budgeted policy engine, delivered as a Model Context Protocol (MCP) proxy that requires no change to the agent.
3. A pre-registered evaluation protocol on tau-bench [3] and SWE-bench [4] against six reference policies, with the claim under test stated as Pareto dominance over confidence gating rather than as an accuracy figure.
4. A synthetic analysis that quantifies the size of the gap between consequence-aware and confidence-based gating under explicit assumptions, and that identifies silent, compositional irreversibility as the factor bounding any per-call estimator.

The remainder of this paper is organized as follows. Section II reviews related work. Section III formalizes the decision problem. Section IV describes the system. Section V specifies the evaluation protocol. Section VI reports the synthetic

analysis. Section VII discusses the findings, Section VIII states threats to validity, and Section IX concludes.

II. Related Work

A. Agent benchmarks and agent safety evaluation

Our work draws on, and departs from, three bodies of research. The first is capability benchmarks. WebArena [1], tau-bench [3], and SWE-bench [4] place agents in realistic environments with tools that can modify state, which is why we adopt them as our testbed. Their scoring, however, treats every failure alike: an agent that abandons a task and an agent that erases the user's data both receive zero. These benchmarks measure whether a task was completed, not how much harm resulted when it was not.

The second is agent safety evaluation, which is closer to our concerns but classifies actions rather than measuring them. ToolEmu [5] uses a language model to emulate a sandbox and elicit risky behavior. Other benchmarks test whether models recognize risk in interaction records or resist prompt injection, and runtime monitors go further and permit or block actions according to their type. This work answers the question "is this action risky?", typically with a coarse label and often after the action has occurred. None of it estimates what the action would cost to reverse.

The third, and the closest to our approach, is AI control research on trusted monitoring, in which a weaker but trusted model oversees a stronger untrusted one under a limited auditing budget. We adopt this budget structure almost unchanged. The difference lies in what triggers an audit: in trusted monitoring it is suspicion of misbehavior, whereas in our setting it is the measured cost of undoing the action.

B. Reversibility in reinforcement learning

The irreversibility of some actions is not a new concern; reinforcement learning has studied it for years. Some methods penalize side effects by tracking which states remain reachable after an action, others preserve the agent's ability to pursue many possible goals, and still others learn to estimate reversibility in a self-supervised manner [6]. All of these methods share the same foundations: a well-defined state space, a model of how actions move between states, and an environment that can be reset.

Without those foundations the methods no longer apply. An agent calling a payments API has no explicit state space to reason over, reachability cannot be computed, and a real bank transfer cannot be reset. How difficult an action is to undo depends on events in the external world, not on the rules of a simulator. We therefore retain the idea and set aside the machinery: the intuition behind reachability still guides our design, but instead of computing undo cost from a model of the environment, we will measure it directly by executing actions in a sandbox and recording what is required to reverse them.

C. Uncertainty-aware escalation and the cost of interruption

The closest framing to ours is uncertainty alignment. KnowNo [2] uses conformal prediction to let an LLM planner recognize when to ask for help: when its set of plausible next steps is ambiguous, it stops and asks. Our rule also requires a calibrated error probability, but such a probability is not sufficient on its own. A 10% chance of error may justify proceeding with a harmless action and halting a costly one, and the probability alone cannot distinguish the two cases. For calibration itself we draw on conformalized quantile regression [7], applied in Section IV-C.

We also build on two older fields that agent research has largely overlooked. The first is distributed databases, where sagas and compensating transactions [8] provide precise terms for undoing work: an exact inverse, a compensating action that offsets the original, or a partial restoration. The rollback axis of our taxonomy is built on this vocabulary. The second is human-computer interaction. Studies of interruption timing [9] show that every interruption has a cost, that this cost depends on when and how often interruptions occur, and that batching interruptions reduces it. A final thread is decision-focused learning, which argues that a model should be trained on the cost of the decisions it produces rather than on the accuracy of its predictions. This is the appropriate objective for our policy, yet to our knowledge it has not been applied to agent supervision under an interruption budget.

D. Concurrent work on tool-call reversibility and escalation

Several recent preprints, most of them concurrent with this work, address parts of the same problem. POLAR [10] is the closest. Before a mutating tool call executes, it searches a hand-authored, per-domain action ontology for an inverse sequence, grounded in a state projection rebuilt from the executed tool record, and scores the result by its weakest step. Each step is scored through one of five typed relations, ranging from exact inverses through compensation to fail-closed irreversibility and weighted by curator-assigned confidence, a gradient that our rollback axis shares. A pluggable policy then vetoes the call, asks the user for consent, or only logs it. POLAR differs from Guardrail in two respects. First, its score measures annotated recoverability rather than cost: the inverse sequence is, in the authors' words, a structural hypothesis that is never executed, and no monetary or latency terms enter it. Second, its threshold is fixed and its escalation is consent-based, with no estimate of error probability and no interruption budget. The authors suggest, as an untested direction, invoking the structural check only when an uncertainty signal is high; the rule in (2) combines the two quantities differently, multiplicatively rather than as a routing condition.

Other recent systems either recover after the fact or classify risk rather than predicting undo cost in advance. Transactional agents synthesize compensating actions and invoke them on failure, and runtime interceptors return allow, warn, block, or review verdicts from rules and an LLM judge.

On escalation, DosSantos DiSorbo and Ju [11] study language models that compare the expected costs of acting and escalating, which is structurally the decision rule in (2), but with cost ratios fixed per experiment rather than estimated for each action. Liu et al. [12] share our use of counterfactual execution: at 1,036 ToolSandbox decision points they clone the environment, execute each of four meta-actions (ask, search, act reversibly, commit irreversibly) for one turn, and let the agent complete the episode. A regressor trained on these outcomes selects the meta-action, scoring each by task success minus interaction cost minus a critical-error indicator. Their labels measure downstream task outcomes rather than undo cost; irreversibility is a property of the action category, and the critical-error flag is reported by the environment rather than measured by a rollback; the scores are not calibrated; and each decision is labeled independently, so the cumulative effect of successive actions is not captured.

Taken together, these lines of work narrow the contribution of this paper. Pre-execution reversibility scoring with consent-based escalation [10], expected-cost escalation [11], and outcome labels from counterfactual execution [12] each exist separately. To our knowledge, no published system combines them: a continuous undo-cost estimate for an arbitrary tool call, measured by executing rollbacks rather than derived from annotations, bounded with distribution-free calibration, combined with an error probability in an expected-damage rule under an explicit interruption budget, and extended to irreversibility that arises only across sequences of calls. This combination, rather than any single component, is the contribution Guardrail is designed to make, and its value must be established empirically against the strongest of these approaches.

III. Problem Formulation

A. Decision rule

Let a denote a proposed tool call, s the relevant environment state, and h the trajectory history. Let $P_{err}(a, s, h)$ be the probability that the call is wrong with respect to user intent, $U(a, s, h)$ the cost of undoing it if it is, and $C_{int}(h)$ the cost of interrupting the operator at this point in the trajectory. The expected damage of executing without review is

$$D(a) = P_{err}(a, s, h) \cdot U(a, s, h), \tag{1}$$

and the gate escalates when

$$P_{err} \cdot \hat{U}^{+} > \lambda(b) \cdot C_{int}(h), \tag{2}$$

where $\hat{U}^{+}$ is a calibrated upper bound on undo cost (Section IV-C) and $\lambda(b) \geq 1$ is a multiplier that increases as the remaining interruption budget b is depleted, so that the effective threshold rises late in a long task rather than starving it. Undo cost is decomposed into recovery latency L, monetary exposure M, and social exposure S:

$$U = w_L L + w_M M + w_S S. \tag{3}$$

The weights are not claimed to be known. Every headline result in the evaluation is reported over a grid of weight settings rather than at one point (Section V).

B. Why a confidence threshold is insufficient

The difference is clearest geometrically. Consider each proposed action as a point with the agent's confidence c on one axis and its undo cost U on the other. A confidence gate escalates whenever $c < t$; if confidence is calibrated, this is equivalent to escalating whenever $P_{err} > 1 - t$. In this plane the gate selects everything to the left of a single vertical line, and undo cost plays no part in the decision.

The rule in (2) draws a different boundary. It escalates whenever $(1 - c)\, U > C$, a region bounded by the hyperbola $(1 - c)\, U = C$. Low-confidence actions may still proceed if they are cheap to reverse, and high-confidence actions may still be stopped if they are expensive.

The two boundaries agree only in a special case: when all actions lie along a single monotone curve, so that an action's confidence determines its undo cost. In every other case there exist pairs of actions for which the confidence gate inverts the correct decision, escalating the cheap action and executing the costly one, at every threshold t. Raising t to catch a high-confidence irreversible action also escalates every recoverable action with lower confidence; lowering t to grant the agent more autonomy lets the irreversible action through. Fig. 1 illustrates this on one sample from the synthetic population described in Section VI.

The argument is elementary, but it changes the question the evaluation must answer. The aim is not to show that a consequence-aware gate is more accurate, but that it reaches combinations of error rate and interruption rate that no confidence threshold, at any setting, can reach.

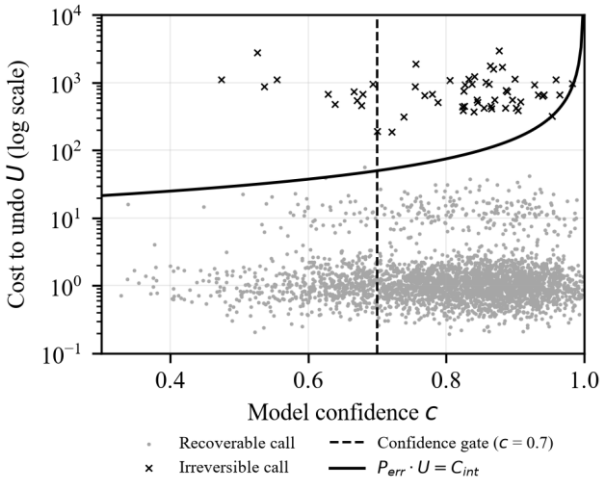


Figure 1: Synthetic actions in the confidence–undo-cost plane. The vertical confidence gate cannot separate irreversible calls (crosses) from recoverable ones; the expected-damage boundary (solid) can.

IV. System Design

This section describes the design of Guardrail. Implementation is in progress: none of the components below has yet been built or evaluated, and they are described as they will be implemented. Guardrail will be interposed between an agent and its tools. Every proposed call will be intercepted before execution and passed through three components: an undo-cost regressor, a detector for contextual and compositional irreversibility, and a policy engine that returns *execute* or *escalate*. The components will be trained on labels produced by a sandbox organized around an irreversibility taxonomy.

A. Irreversibility taxonomy

Actions are classified along four axes (Table 1). The rollback axis follows the compensating-transaction vocabulary of [8]. The taxonomy covers three domains chosen because they span the range of undo cost and because automated rollback verification is feasible for each: file system, source control and deployment, and financial or transactional operations against mock services with synthetic records. It will be released as a specification with a JSON schema and validated by inter-annotator agreement on 200 actions.

Table 1: Axes of the Irreversibility Taxonomy

Axis	Values	Source of label
Domain	File system; source control and deployment; financial / transactional	Tool metadata
Rollback	Exact inverse; compensating action; partial restoration; none	Measured in sandbox
Externality	Contained; crosses organizational boundary; reaches third party	Assigned
Latency sensitivity	Rate at which reversibility decays after execution	Measured / simulated

B. Rollback-instrumented sandbox

Every action that enters the sandbox will follow the same procedure. Before execution, the sandbox will snapshot everything the action could affect: a hash tree of the file system, a database dump, the state of a mock ledger, and the outbound message queue. The action will then be executed, and the sandbox will attempt to reverse it using the rollback mechanism prescribed by the taxonomy, within a bounded time budget. Finally, the result will be compared against the original snapshot; any residual difference is the part of the state that was not restored.

Two cost components follow directly from this procedure. Recovery latency L is the time the rollback required, and monetary exposure M is read from the ledger. The third component, social exposure S, cannot be measured in this way; no experiment quantifies the reputational cost of an incorrect email reaching a client. We will therefore assign S from the externality axis of the taxonomy. This is a modeling assumption, and it will be labeled as such in our results.

Some events that make an action irreversible lie outside the agent's control, such as a recipient opening a message or a collaborator pulling a faulty commit. Rather than encoding these with fixed rules, we will simulate them as stochastic processes whose parameters are documented and published, so that others can inspect or modify them. The target output is an open dataset of at least 10,000 labeled actions.

C. Undo-cost regressor

Inputs are structural features of the call (tool name, argument types, destructive flags, wildcard scope), a sentence-encoder embedding of the call that generalizes to unseen tool names, and state features such as target sensitivity and whether the target is a production resource. The two directions of error are not symmetric: under-prediction permits a destructive action, whereas over-prediction merely spends interruption budget. The regressor will therefore be trained with the pinball loss at a quantile $\tau > 0.5$,

$$\ell_\tau(y, \hat{y}) = \max(\tau(y - \hat{y}), (\tau - 1)(y - \hat{y})). \quad (4)$$

Lower and upper quantile heads give an interval, and split conformal calibration in the form of conformalized quantile regression [7] widens it to a distribution-free bound with marginal coverage $P(U \le \hat{U}^+) \ge 1 - \alpha$ on exchangeable data. The policy will consume only the upper end $\hat{U}^+$.

The error probability P_{err} will combine self-consistency across sampled rollouts with a calibrated classifier. Because sampling multiplies agent cost, the gate will be staged: a cheap feature screen will resolve calls that are clearly safe or clearly catastrophic, and sampling will run only where the interval straddles the threshold. The added latency and token overhead will be measured, not assumed away.

D. Silent-irreversibility detector

Some irreversibility cannot be detected by examining one call at a time. Suppose an agent deletes a backup. In isolation this is recoverable, because the primary record remains. Later, it overwrites the primary record. In isolation this is also recoverable, because the record could be restored from the backup. Each step leaves a way back, and a per-call estimator would correctly score both as cheap; executed in sequence, however, they leave nothing to restore from. The detector will address such cases with two heads on top of the cost regressor.

The contextual head will consider the trajectory so far, together with any observed external events, and predict a factor by which the regressor's cost estimate should be scaled.

The compositional head will track the remaining recovery paths. It will maintain a typed resource graph recording which recovery affordances remain available for each resource (primary copy, backup, snapshot, audit copy), and score each proposed action by the affordances it would consume. In the example above, by the time the overwrite is proposed the graph already records the backup as deleted, and the overwrite will be priced accordingly. An action that removes the last remaining recovery path will therefore be amplified sharply, even when the action itself appears harmless. This head will be trained on a fixed set of composition patterns, each executed as a full sequence in the sandbox to confirm that it leaves no recoverable state, which reuses the labeling loop of Section IV-B without additional infrastructure.

E. Escalation policy engine

The engine will learn a threshold per action class by minimizing realized total cost, namely damage from unescalated errors plus interruption cost. Escalations will be capped as a rate per hundred actions rather than a count per task, so long tasks are not starved; $\lambda(b)$ in (2) raises the effective threshold as the budget depletes; and non-blocking escalations in close succession will be batched into one review, following the interruption literature [9]. Reviewer accuracy will be treated as a parameter below one rather than as a perfect oracle. Every escalation, verdict, and triggering feature will be written to an append-only audit log.

F. Integration surfaces

The primary artifact will be an MCP proxy that sits between the agent and its tool servers and requires only an endpoint change. A Python SDK will expose a decorator and a context manager that wrap any tool function in a few lines; because interception happens at the function-call boundary shared by mainstream agent frameworks, one implementation will cover all of them. A minimal console will provide the review inbox, a per-task risk timeline, and the audit log.

V. Evaluation Protocol

A. Baselines

Guardrail will be compared against the six reference policies in Table 2. B2+ is included so that confidence gating is not disadvantaged by having fewer free parameters than a per-class policy. All policies will be swept across their full threshold range under an equal tuning budget. Since neither paper states a code release, the veto and escalate policies of POLAR [10] and the controller of Liu et al. [12] will be reimplemented from their published descriptions, added as further baselines, and reported as adaptations.

Table 2: Reference Policies

ID	Policy	Escalation rule and role
B1	No gating	Never escalates; upper bound on irreversible errors
B2	Confidence gate	One global threshold on confidence; dominant practice, primary comparison
B2+	Per-domain confidence	One confidence threshold fitted per domain
B3	Entropy / self-consistency	High predictive entropy or low agreement across sampled rollouts
B4	Static allow/deny list	Membership in a hand-written danger list; current industrial guardrails
B5	Zero-shot LLM judge	A frontier model rates undo cost; the rating is thresholded
B6	Hindsight oracle	Escalates exactly the erroneous actions; unachievable limit

B. Metrics

The primary metric is the irreversible-error rate: the proportion of tasks containing at least one erroneous executed action that left a state which could not be restored. Correct actions that are intentionally irreversible are excluded. An action counts as erroneous when it diverges from the benchmark reference solution or violates its policy checks, with these criteria fixed per environment before experiments begin. Secondary metrics are interruptions per task, realized damage cost, task success rate (as a control against safety by refusal), gate overhead as median and 99th-percentile latency together with the token-cost multiplier, and regression quality of the undo-cost estimator.

C. Pre-registered claims and ablations

The headline result is a Pareto frontier relating irreversible-error rate to interruption frequency. The pass condition is a statistically significant reduction in irreversible-error rate at matched interruption frequency against both B2 and B2+, a significant reduction in interruptions at matched error rate, and no loss of task success relative to B1. A 40% relative reduction is stated as a target, not as a pass threshold. Significance will be assessed with paired bootstrap and Wilcoxon signed-rank tests over five seeds, and generalization will be tested on a held-out domain. Ablations will remove, in turn, the detector, the conformal calibration, the learned thresholds, and the budget. The primary ablation fixes P_{err} at a constant, isolating gating on consequence alone from the quality of confidence estimation: if that variant already dominates B2, the claim holds independently of how error probability is estimated.

VI. Synthetic Analysis

The empirical study above has not yet been run; the sandbox and dataset are under construction. Before committing to it, a synthetic analysis was carried out to estimate the size of the effect the protocol is designed to detect and to identify what limits it. The numbers in this section characterize the decision rule under stated assumptions and must not be read as measurements of the system.

A. Setup

The population contains 20,000 tasks of 20 actions each. Confidence c follows Beta(8, 2) (mean 0.80) and the error probability is $P_{err} = 1 - c$, i.e. confidence is assumed perfectly calibrated, which favors B2. Undo cost follows a three-part log-normal mixture: 90% cheap actions (log-mean 0), 8% costly but recoverable actions (log-mean 2.5), and 2% effectively irreversible actions (log-mean 6.5); an action is irreversible if $U \geq 100$. The gate observes not U but a noisy estimate $\hat{U} = U \cdot \varepsilon$ with $\log \varepsilon \sim N(0, 1)$. To model silent irreversibility, 25% of irreversible actions are given an estimate drawn from the cheap component, so that they look harmless in isolation. A human reviewer catches an escalated error with probability 0.9. Two dependence settings are reported: confidence independent of cost, and a Gaussian copula with $\rho = -0.3$ under which costly actions also tend to be low-confidence, which again favors B2. Each policy is swept over its threshold range and its irreversible-error rate is read off at fixed budgets of interruptions per task.

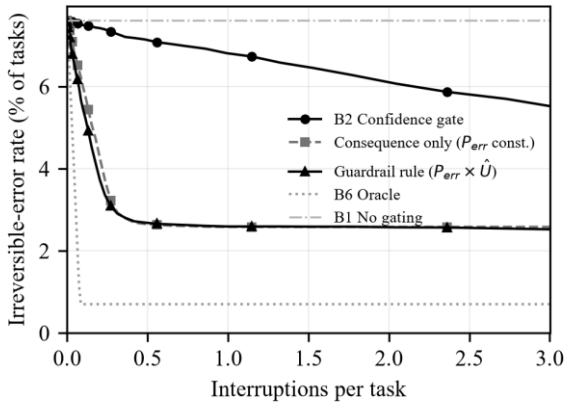


Figure 2: Error–interruption frontiers on the synthetic population (independent setting). Lower-left is better.

Table 3: Irreversible-Error Rate (% of Tasks) at Matched Interruption Budget, Synthetic Population

Policy	0.25/task	0.5/task	1/task	2/task
Independent				
B1 No gating	7.61	7.61	7.61	7.61
B2 Confidence	7.37	7.13	6.81	6.10
Consequence only	3.51	2.65	2.58	2.58
Guardrail rule	3.25	2.68	2.60	2.58
B6 Oracle	0.70	0.70	0.70	0.70
Correlated ($\rho = -0.3$)				
B1 No gating	11.30	11.30	11.30	11.30
B2 Confidence	10.22	9.53	8.50	6.96
Consequence only	5.24	3.79	3.72	3.72
Guardrail rule	4.90	3.79	3.73	3.71
B6 Oracle	1.15	1.15	1.15	1.15

B. Observations

Three observations follow from Table 3 and Fig. 2. First, at a budget of one interruption per task the expected-damage rule lowers the irreversible-error rate from 6.81% to 2.60% of tasks in the independent setting (a 61.9% relative reduction) and from 8.50% to 3.73% in the correlated setting (56.1%), with a perfectly calibrated confidence gate as the comparison. In the independent setting this outcome is close to true by construction, since a confidence gate cannot see a quantity independent of confidence; its value lies in the magnitude, and in the fact that a moderate correlation in favor of the baseline does not close the gap. Confidence gating does improve as its budget grows, but at two interruptions per task it still sits at 6.10%, far above the consequence-aware curves at a quarter of that budget.

Second, the consequence-aware curves flatten at approximately 2.58%, well above the oracle floor of 0.70% set by reviewer error. Additional budget does not help, because the residual errors are the silent irreversible actions whose estimated cost is drawn from the cheap component; no threshold on a per-call estimate reaches them. This is the quantitative motivation for the compositional detector of Section IV-D, and it identifies the detector, not the regressor, as the component most likely to bound real-world performance.

Third, the consequence-only variant with P_{err} held constant is within 0.27 percentage points of the full rule at every budget. Under these assumptions almost all of the separating signal comes from the consequence term. Whether that holds for real agents, whose confidence may carry more information about errors on costly actions than a Beta prior does, is precisely what the primary ablation of Section V-C is designed to test.

VII. Discussion

At its core, this paper makes one point: confidence and consequence answer different questions, and a gate that decides when to involve a human needs both answers. Confidence gating is not wrong; it is incomplete in a way that no threshold tuning can repair, because its decision boundary is perpendicular to the direction in which damage varies. The synthetic analysis quantifies this gap.

Two design decisions deserve explanation. First, Guardrail favors escalation when it is uncertain about cost; both the asymmetric loss and the conformal upper bound push it in that direction. This choice is deliberate. An unnecessary interruption is a minor, recoverable cost, whereas an unescalated irreversible error is not. A consequence of this choice is that the interruption budget, rather than the estimator, determines how cautious the system is in practice. The budget and its depletion schedule are therefore central evaluation parameters rather than settings to be tuned at the end.

Second, enforcement is placed in an MCP proxy rather than in the agent's prompt. This targets precisely the weakness described in Section I: a rule written into a prompt depends on the model choosing to follow it, whereas a rule in the execution path does not require the model's cooperation.

The evaluation is also falsifiable: the central claim fails if the consequence-only ablation does not dominate B2+, or if gate latency makes the system impractical. If real undo costs prove predictable from a short list of tool names, B4 will match Guardrail, and we will report that as a finding.

VIII. Threats to Validity and Limitations

First, and most importantly, no empirical results are reported in this paper. All quantitative findings come from a synthetic population whose distributions were chosen by the authors; they bound the plausible size of the effect under those assumptions and should not be cited as evidence of system performance. Second, sandbox undo cost may diverge from production undo cost within an action class; this will be mitigated by validating a stratified subsample of labels against expert judgment, but it cannot be eliminated. Third, social exposure is assigned from the externality axis rather than measured, and the weights in (3) are a modeling choice; results will be reported over a weight grid for this reason. Fourth, high-cost actions are rare in natural agent trajectories, so the dataset will rely on deliberate synthesis and cost-weighted sampling, which may not match the natural distribution of risky calls. Fifth, conformal coverage holds marginally under exchangeability, which agent trajectories need not satisfy, particularly on a held-out domain. Sixth, the system gates before execution only, analyses one agent executing one task, and does not address adversarial prompt injection, which may deliberately disguise an irreversible call as a cheap one. Finally, the reviewer is modeled as reachable and imperfect but not as fatigued. Reviewer fatigue may make safety non-monotonic in the escalation rate; the interruption budget is intended to guard against this, but this study does not measure the effect directly.

IX. Conclusion

This paper set out to change the question an agent's safety gate asks before it acts. Most gates ask how confident the agent is. Guardrail asks what it would cost to undo the action if the agent were wrong, and pauses for a human when that expected cost is too high.

We have committed in advance to evaluating Guardrail against six baselines on tau-bench and SWE-bench, and the claim under test is that it reaches regions of the error–interruption plane that no confidence threshold can reach. The synthetic analysis, with its assumptions stated explicitly, suggests that the difference is substantial: at the same interruption budget, irreversible errors fall by roughly half or more. It also shows where the approach reaches its ceiling, at irreversibility that emerges only when several individually harmless calls are combined.

Four directions remain for future work: (i) completing the sandbox and releasing the labeled undo-cost dataset; (ii) running the full evaluation and reporting the frontier with its ablations; (iii) extending the taxonomy to communication and network domains; and (iv) studying two risks the current design does not address, namely reviewer fatigue, in which escalations are approved without real scrutiny, and adversaries who disguise irreversible calls as harmless ones.

References

[1] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "WebArena: A Realistic Web Environment for Building Autonomous Agents," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[2] A. Z. Ren *et al.*, "Robots That Ask For Help: Uncertainty Alignment for Large Language Model Planners," in *Proc. Conf. Robot Learning (CoRL)*, PMLR, vol. 229, 2023.

[3] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains," arXiv:2406.12045, 2024.

[4] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" in *Proc. ICLR*, 2024.

[5] Y. Ruan *et al.*, "Identifying the Risks of LM Agents with an LM-Emulated Sandbox," in *Proc. ICLR*, 2024.

[6] N. Grinsztajn, J. Ferret, O. Pietquin, P. Preux, and M. Geist, "There Is No Turning Back: A Self-Supervised Approach for Reversibility-Aware Reinforcement Learning," in *Proc. NeurIPS*, 2021.

[7] Y. Romano, E. Patterson, and E. Candès, "Conformalized Quantile Regression," in *Proc. NeurIPS*, 2019, pp. 3543–3553.

[8] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 1987, pp. 249–259.

[9] P. D. Adamczyk and B. P. Bailey, "If Not Now, When? The Effects of Interruption at Different Moments Within Task Execution," in *Proc. ACM CHI*, 2004, pp. 271–278.

[10] Y. Kang *et al.*, "POLAR: Ontology-Guided Risk Prevention for Tool-Calling LLM Agents," arXiv:2610.08082, 2026.

[11] M. DosSantos DiSorbo and H. Ju, "Act or Escalate? Evaluating Escalation Behavior in Automation with Language Models," arXiv:2604.08588, 2026.

[12] Q. Liu *et al.*, "Agents' Last Doubt: Knowing When to Act and When to Step Back in Tool-Using Agents," arXiv:2610.24194, 2026.